SYSTEM DESIGN & PROJECT PROPOSAL

Predictive Autoscaling Layer for Kubernetes

Forecast-driven, Kubernetes-native capacity management

A Kubernetes-native predictive scaling system that forecasts future workload and proactively prepares application and cluster capacity before demand spikes occur.

PROJECT DOMAIN

Cloud Infrastructure, Kubernetes, Machine Learning

PRIMARY FOCUS

Predictive autoscaling and proactive capacity management

TEAM

Sahil Kukreja & Tanay Shukla

Purpose: To define the problem, proposed solution, technology stack, functional requirements, non-functional requirements, and team responsibilities for mentor/faculty review.

1 Problem Statement

Kubernetes applications commonly use reactive autoscaling mechanisms such as the Horizontal Pod Autoscaler (HPA). HPA increases or decreases application replicas after resource utilization or another scaling metric reaches a configured target. This works for normal workload variation, but it does not anticipate predictable future demand.

For workloads with recurring traffic patterns — such as daily morning peaks, month-end payment processing, scheduled marketing campaigns, product launches, or known business events — waiting for demand to increase before scaling introduces a delay. New pods need to be scheduled, and additional cluster nodes may also need to be provisioned. During this period, applications can experience higher latency, request queuing, or dropped requests.

> This project proposes a predictive autoscaling layer that uses historical time-series metrics from Prometheus and optional external signals to forecast future workload. The system proactively prepares Kubernetes capacity before the predicted demand arrives, while continuing to work with existing Kubernetes autoscaling mechanisms.

2 Description

The proposed system will operate as a Kubernetes-native predictive scaling layer. Prometheus will provide historical workload and resource metrics such as request rate, CPU utilization, memory utilization, latency, and pod information. A machine-learning forecasting service will process these time-series data and estimate workload during a configurable future window.

The initial forecasting implementation will use Prophet because the problem contains time-based patterns and seasonality, and Prophet provides a practical forecasting approach with support for additional regressors. The ML component will be periodically retrained using newly collected historical data. The design will remain model-agnostic so that models such as LSTM or other time-series approaches can be evaluated later.

The forecast will be converted into a scaling recommendation. A Go-based Kubernetes operator using controller-runtime will act as the control layer. It will watch a Custom Resource Definition (CRD), evaluate predictions and policies, and reconcile the desired scaling state through the Kubernetes API.

A central engineering challenge is coexistence with HPA. The predictive layer must avoid repeatedly fighting with HPA over replica count. The design will therefore use explicit coordination, confidence thresholds, cooldown periods, scaling limits, and controlled restoration of the normal HPA configuration. The final ownership strategy will be validated during implementation.

Where Karpenter is available, the operator will also support proactive preparation of node capacity when predicted pod demand is expected to exceed the current cluster capacity. This addresses the delay between requesting additional pods and having enough nodes available to schedule them.

Proposed End-to-End Flow

COMPONENT	RESPONSIBILITY
Prometheus	Collects and stores historical Kubernetes/application metrics.

ML Prediction Service	Prepares metrics, trains/retrains the model, and forecasts future workload.
Decision Layer	Checks forecast confidence, scaling limits, cooldowns, and cost guardrails.
Go Kubernetes Operator	Reconciles predictive scaling state and communicates with Kubernetes APIs.
HPA Coordination	Coordinates proactive scaling with reactive autoscaling to reduce thrashing.
Karpenter	Prepares additional node capacity when predicted demand requires it.
Recovery	Returns the system to normal reactive scaling after the predictive window.

3 Tech Stack

LAYER	TECHNOLOGY	PURPOSE
Orchestration	Kubernetes	Deployment, scheduling, networking and scaling.
Operator	Go + controller-runtime	CRDs, reconciliation loops and Kubernetes control logic.
Configuration	Kubernetes CRD	Defines predictive scaling policies for workloads.
Metrics	Prometheus	Historical and real-time time-series metrics.
ML	Python + ML Model	Forecast future workload.
Data Processing	Python + Pandas	Clean and transform metric data.
Containers	Docker	Package operator and ML services.
Local Testing	Minikube	Local Kubernetes development environment.

4 Functional Requirements

FR-01 **Metric Collection**

Collect historical workload and resource metrics from Prometheus for configured Kubernetes workloads.

FR-02 **Metric Preparation**

Clean, transform and aggregate time-series data into a format suitable for forecasting.

FR-03 **Forecast Generation**

Generate a forecast for a configurable future time window using the selected forecasting model.

FR-04 **Periodic Model Retraining**

Support periodic retraining using newly available historical data.

FR-05 **Confidence-Aware Decisions**

Use forecast confidence and a configurable confidence threshold before proactive scaling.

FR-06 **Replica Recommendation**

Estimate required application capacity subject to configured minimum and maximum limits.

FR-07 **Kubernetes Integration**

Communicate with the Kubernetes API and apply approved predictive scaling actions.

FR-08 **HPA Coordination**

Coordinate predictive scaling with HPA to reduce conflicting writes and scaling thrashing.

FR-09 **Node Pre-Warming**

Support proactive node capacity preparation through Karpenter when required.

FR-10 **Automatic Recovery**

Restore the normal scaling configuration after the predictive window or when a prediction becomes invalid.

FR-11 **Safety Limits**

Enforce minimum/maximum replicas, cooldown periods and cost/resource guardrails.

FR-12 **Observability**

Expose/log prediction decisions, scaling actions, confidence values and errors.

FR-13 **Kubernetes-Native Configuration**

Provide a CRD for target workload, forecast window, limits and scaling policies.

5 Non-Functional Requirements

NFR-01 **Reliability**

Continue operating safely when predictions fail, metrics are temporarily unavailable, or external signals cannot be reached.

NFR-02 **Safety**

Never scale beyond configured limits; stale, invalid or low-confidence predictions must not cause uncontrolled scaling.

NFR-03 **Performance**

Complete prediction and decision processing sufficiently ahead of the predicted demand window.

NFR-04 **Scalability**

Support multiple workloads and allow additional forecasting models or signal sources without redesigning the operator.

NFR-05 **Availability**

Allow production-oriented deployment with highly available operator and ML components.

NFR-06 **Observability**

Make prediction results, scaling decisions, events and failures traceable through logs and metrics.

NFR-07 **Security**

Use least-privilege Kubernetes RBAC and restrict access to required resources.

NFR-08 **Maintainability**

Keep the ML service and Kubernetes control layer modular so they can be tested and upgraded independently.

NFR-09 **Cost Efficiency**

Avoid unnecessary pre-scaling and enforce configurable cost/replica guardrails.

NFR-10 **Extensibility**

Permit future forecasting algorithms, external signals, scaling policies and cloud integrations.

6 Roles

TEAM MEMBER	ROLE	RESPONSIBILITIES
Sahil Kukreja	KUBERNETES / INFRASTRUCTURE & OPERATOR	Design and implement Kubernetes architecture; develop the Go controller using controller-runtime; define CRDs; integrate Kubernetes APIs and HPA; implement reconciliation, scaling policies, safety guardrails and rollback; integrate Karpenter for node pre-warming; package/deploy with Docker and Helm; perform Kubernetes integration and end-to-end infrastructure testing.
Tanay Shukla	MACHINE LEARNING / FORECASTING	Design and implement the forecasting pipeline; collect and prepare Prometheus time-series data; develop and evaluate the Prophet model; implement retraining; investigate external signals and regressors; evaluate prediction confidence and accuracy; provide prediction outputs to the Kubernetes control layer.

Collaboration boundary: Tanay owns the ML forecasting pipeline and prediction quality, while Sahil owns Kubernetes orchestration, operator behavior and infrastructure integration. The interface between these responsibilities should be a versioned prediction/decision contract containing the target workload, forecast horizon, predicted demand, confidence and recommended capacity.

7 Proposed Initial Scope

The first working prototype will focus on a single Kubernetes workload, Prometheus-based historical metrics, Prophet forecasting, a Go-based Kubernetes operator, HPA coordination and controlled replica pre-scaling.

The system will initially be validated locally using Minikube and a test workload. Karpenter integration, external signal ingestion, cost-aware policies and multi-service forecasting can then be added as subsequent capabilities. A production-oriented deployment can target AWS EKS.

Evaluation should measure forecast accuracy, scaling lead time, scaling stability, resource/cost impact and whether proactive scaling reduces the performance degradation associated with reactive scaling.